

YOLOv11 Model Training

YOLOv11 Model Training

1. Course Content
2. Pre-trained Models
3. Training Traffic Sign Recognition Model (Case 1)
 - 3.1 Start Training
4. Training Lane Detection Model (Case 2)
5. Source Code Analysis
 - 5.1 YOLOv11 Training Program

[!IMPORTANT]

- The factory image already includes a pre-trained traffic sign recognition model. This tutorial is provided for users who need to train their own YOLOv11 object detection models and master model training methods.
- If you want to experience the complete functionality directly, you can skip this section!!!
- If using the dataset from the factory image to train the model, you need to first decompress the .zip format dataset compressed package.
- This tutorial is mainly conducted on the Orin Nano board

1. Course Content

1. Master the method of training YOLOv11 object detection models

2. Pre-trained Models

- **Only Orin boards are recommended for on-board model training. RDK boards have insufficient performance and require annotation and training on a host with GPU. The training source code is in the yolo_train folder under Model Training in the source code summary. If training is required, you can place the yolo_train folder from the model training compressed file in the attachment into the root directory of the main board.**
- yolo11n.pt base model

```
/home/jetson/yolo_train/yolo11n.pt
```

```
jetson@yahboom:~/yolo_train$ ls
conver_model.py  train_lane  train.py  train_sign  yolo11n.pt
jetson@yahboom:~/yolo_train$
```

- Lane detection model path:

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/lane_v6.engine
```

- Traffic sign recognition model path:

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/sign_model.engine
```


- During the training process, the training duration and loss value for each round will be displayed

```
jetson@yahboom: ~/yahboomM3_ws/yolo_train
jetson@yahboom: ~/yahboomM3_ws/yolo_train151x45
train: Fast Image access (ping: 0.0±0.0 ms, read: 87.2±37.6 MB/s, size: 36.6 KB)
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 88 images, 1 backgrounds, 0 corrupt: 28%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 180 images, 4 backgrounds, 0 corrupt: 58%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 283 images, 4 backgrounds, 0 corrupt: 91%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
t/s 0.3s
train: New cache created: /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels.cache
val: Fast Image access (ping: 0.0±0.0 ms, read: 104.1±24.4 MB/s, size: 44.4 KB)
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 88 images, 1 backgrounds, 0 corrupt: 28%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 166 images, 4 backgrounds, 0 corrupt: 53%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 298 images, 4 backgrounds, 0 corrupt: 96%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
t/s 0.3s
val: New cache created: /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels.cache
/home/jetson/yahboomM3_ws/.venv/lib/python3.10/site-packages/matplotlib/projections/_init_.py:63: UserWarning: Unable to import Axes3D. This may be due to multiple versions of Matplotlib being installed (e.g. as a system package and as a pip package). As a result, the 3D projection is not available.
warnings.warn("Unable to import Axes3D. This may be due to multiple versions of ")
Plotting labels to /home/jetson/yahboomM3_ws/yolo_train/project/train2/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr' and 'momentum' automatically...
optimizer: AdamW(lr=0.000556, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 6 dataloader workers
Logging results to /home/jetson/yahboomM3_ws/yolo_train/project/train2
Starting training for 50 epochs...

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
1/50 2.2G 1.417 4.511 1.26 46 640: 100% 20/20 0.91t/s 22.4s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 10/10 1.91t/s 5.3s
all 311 724 0.0569 0.0359 0.0456 0.0407

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
2/50 2.31G 1.131 3.525 1.041 18 640: 100% 20/20 1.61t/s 12.2s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 10/10 1.61t/s 6.1s
all 311 724 0.0298 0.546 0.163 0.113

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
3/50 2.31G 1.167 2.58 1.039 31 640: 100% 20/20 1.41t/s 14.2s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 10/10 1.41t/s 7.1s
all 311 724 0.025 0.188 0.271 0.191

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
4/50 2.31G 1.171 2.074 1.05 20 640: 100% 20/20 1.41t/s 14.2s
Class Images Instances Box(P R mAP50 mAP50-95): 70% 7/10 0.91t/s 7.0s<3.2s
```

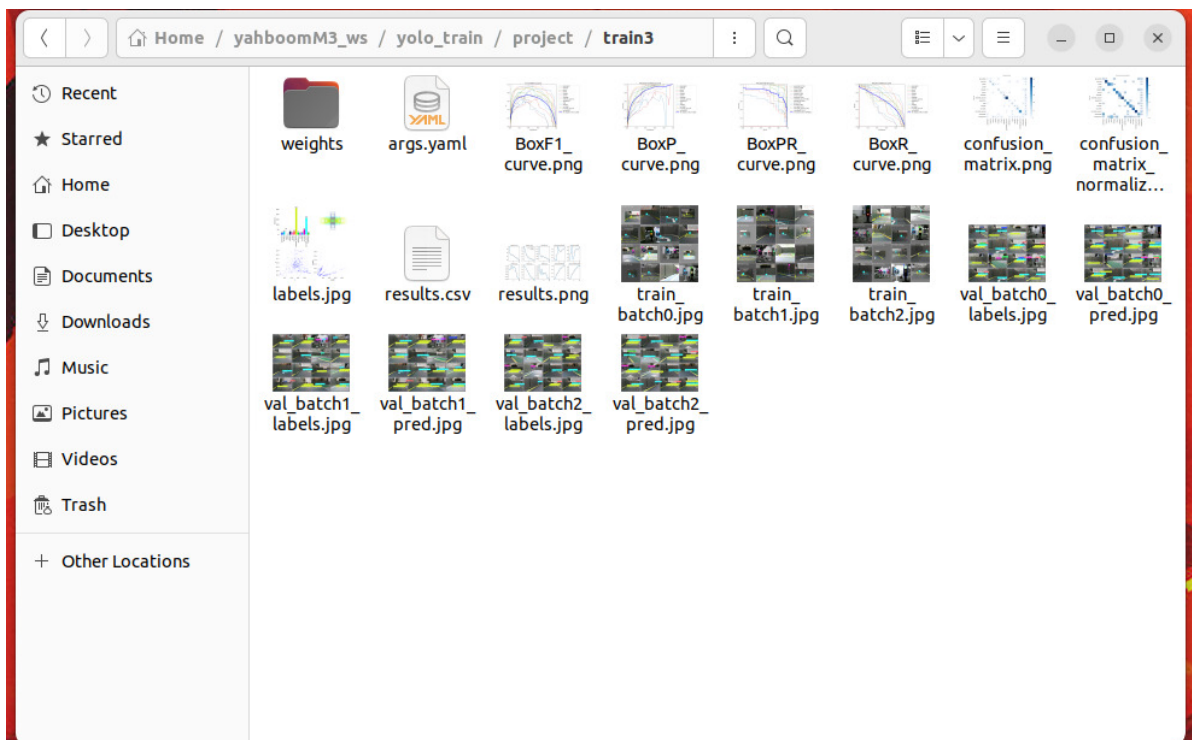
- **Epoch** Current training round
- **GPU_mem** GPU memory currently occupied by training
- **box_loss** Bounding box regression loss value, measuring the difference between predicted boxes and ground truth boxes (position, size). The smaller the value, the more accurate the box positioning
- **cls_loss** Classification loss value, measuring the difference between predicted class and ground truth class. The smaller the value, the more accurate the classification.
- **dfl_loss** Distribution focal loss, used to optimize bounding box regression accuracy, especially for small targets. The smaller the value, the more accurate the bounding box detail prediction.
- **Instances 46**: Total number of target instances contained in the current batch.
- **Size 640**: Input image size is 640x640 (YOLOv11 default training input size).
- After training is completed, the terminal will prompt the save path of the training output files

```

jetson@yahboom: ~/yahboomM3_ws/yolo_train
jetson@yahboom: ~/yahboomM3_ws/yolo_train 110x33
Validating /home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.pt...
Ultralytics 8.3.214 Python-3.10.12 torch-2.5.0a0+872d972e41.nv24.08 CUDA:0 (Orin, 7620MiB)
YOLO11n summary (fused): 100 layers, 2,584,882 parameters, 0 gradients, 6.3 GFLOPs
Class Images Instances Box(P) R mAP50 mAP50-95: 10% 1/1
0 0.5it/s 0.7s<19. Class Images Instances Box(P) R mAP50 mAP50-95: 20
% 2/10 0.7it/s 1.5s<11. Class Images Instances Box(P) R mAP50 mAP50-95: 30%
P50 mAP50-95): 30% 3/10 0.6it/s 4.4s<12. Class Images Instances Box(P) R mAP50 mAP50-95: 40%
R mAP50 mAP50-95): 40% 4/10 0.7it/s 5.6s<9.1 Class Images Instances Box(P) R mAP50 mAP50-95: 50%
tances Box(P) R mAP50 mAP50-95): 50% 5/10 0.9it/s 6.2s<5.3 Class Images Instances Box(P) R mAP50 mAP50-95: 60%
ss Images Instances Box(P) R mAP50 mAP50-95): 60% 6/10 1.1it/s 6.9s<3.5 Class Images Instances Box(P) R mAP50 mAP50-95: 70%
1.4it/s 7.4s<2.2 Class Images Instances Box(P) R mAP50 mAP50-95): 70% 7/10
8/10 1.4it/s 8.0s<1.4 Class Images Instances Box(P) R mAP50 mAP50-95): 80%
0 mAP50-95): 90% 9/10 1.6it/s 8.6s<0.6 Class Images Instances Box(P) R mAP50 mAP50-95): 100%
R mAP50 mAP50-95): 100% 10/10 1.1it/s 9.0s
all 311 724 0.859 0.739 0.817 0.654
greenlight 16 16 0.626 0.525 0.594 0.483
honking 29 29 0.862 1 0.99 0.825
parkA 50 50 0.914 0.637 0.905 0.67
parkB 2 2 1 0 0.0167 0.0117
redlight 30 31 0.754 0.903 0.868 0.68
school 19 19 0.965 0.474 0.82 0.656
sidewalk 34 34 0.918 1 0.995 0.848
sidewalk_ground 165 261 0.965 0.981 0.983 0.636
speedlimit 20 20 0.847 0.6 0.828 0.715
stop 14 14 0.954 0.929 0.974 0.843
straight 16 16 0.871 0.812 0.867 0.759
turnright 51 51 0.977 0.848 0.955 0.825
turnright_ground 164 165 0.91 0.915 0.973 0.664
yellowlight 16 16 0.468 0.715 0.67 0.542
Results saved to /home/jetson/yahboomM3_ws/yolo_train/project/train3

```

- In the output folder we can see the following contents
- **weights** Trained model weight files
- **.png** Score data during training process



4. Training Lane Detection Model (Case 2)

- The training method is the same as before

```
cd ~/yolo_train
```

- Start training

```
python3 train.py --epochs 50 --  
train_config=/home/jetson/yolo_train/train_lane/lane.yaml --project './project2'
```

Here the `--project` parameter specifies the folder for training output files, used to distinguish from the previous traffic sign recognition model. Here the path is set to `'./project2'`. After training is completed, the model weight files will be saved in the path specified by the project parameter

5. Source Code Analysis

5.1 YOLOv11 Training Program

The program source code is located at `/home/jetson/yolo_train/train.py`

- Configuration parameters required during the training process are defined through the `argparse` library, allowing parameters to be passed via command line when running the script
- During the training process, logs, loss curves, evaluation metrics (such as mAP), and model weight files (best.pt, last.pt, etc.) will be automatically generated and saved in the directory specified by `--project`.

```
import argparse  
from ultralytics import YOLO  
  
def get_args():  
    '''  
    ...  
    '''  
    parser = argparse.ArgumentParser()  
    parser.add_argument('--project', default='./project', help='Project file  
output directory', type=str)  
    parser.add_argument('--  
MODEL_PATH', default='/home/jetson/MODELS/yolov11/yolo11n.pt', help='原始模型权重文  
件路径Original model weight file path', type=str)  
    parser.add_argument('--epochs', default=100, help='Training rounds',  
type=int)  
    parser.add_argument('--imgsz', default=640, help='Image size', type=int)  
    parser.add_argument('--train_config', default='', help='Configuration file  
path', type=str)  
    return parser.parse_args()  
  
if __name__ == "__main__":  
    args = get_args()  
    model = YOLO(args.MODEL_PATH) # load a pretrained model  
  
    results = model.train(data=args.train_config, epochs=args.epochs,  
imgsz=args.imgsz, project=args.project) # start training
```

